Manual: como rodar e apresentar o Conecta Vida no Docker Swarm

Projeto **Conecta Vida**, IFSP Bragança Paulista. Repositório: https://github.com/Conecta-Vida/Docker-Teste

Todas as fotos e vídeos deste manual foram feitos com o projeto rodando de verdade num cluster de 3 nós e estão na pasta `documentacao/midia/` do GitHub. A lista completa está na **Parte K**.



Parte A: o que o professor pediu e onde está

Requisito	Como atendemos	Onde mostrar
1. Dockerfiles (imagens) dos serviços	5 imagens em Alpine Linux: `admin-api`, `mobile-api` (Java 21, multi-stage), `admin-web` (React + Nginx), `mobile-web` (Flutter Web + Nginx) e `painel-cluster` (Node.js)	`admin-api/Dockerfile`, `mobile-api/Dockerfile`, `admin-web/Dockerfile`, `mobile-app/Dockerfile` e `painel-cluster/Dockerfile`
2. Docker Swarm com docker-in-docker	Cluster de **3 nós**: o PC é o manager e `worker-1` e `worker-2` são containers `docker:dind`	Visualizer e `docker node ls`
3. Docker Compose (infraestrutura como código), variação de réplicas e Visualizer	`docker-compose.yml` descreve os 7 serviços, a rede, o volume e as réplicas. As réplicas mudam pelo arquivo (`replicas`) e pelo terminal (`docker service scale`)	`docker-compose.yml` e o Visualizer em http://localhost:8085
Diferencial no Visualizer	**Painel do Cluster Conecta Vida**: um visualizador próprio, feito por nós, com o tema do projeto, eventos ao vivo e botões **+ / −** para escalar pela tela	http://localhost:8090

Os 7 serviços da stack:

Serviço	O que é	Porta	Réplicas	Onde roda
`postgres`	Banco de dados	5432	1	Manager (fixo)
`admin-api`	API do painel do gestor	8080	2	Qualquer nó
`mobile-api`	API do app do cidadão	8081	3	Qualquer nó
`visualizer`	Painel gráfico do cluster	8085	1	Manager (fixo)
`admin-web`	Site do painel admin	8088	2	Qualquer nó
`mobile-web`	App do cidadão no navegador	8089	2	Qualquer nó
`painel-cluster`	**Diferencial:** Painel do Cluster Conecta Vida	8090	1	Manager (fixo)

Parte B: preparar o computador (uma vez só)

B1. Instale o **Docker Desktop**: https://www.docker.com/products/docker-desktop/

B2. Instale o **Git for Windows**, que traz o **Git Bash**: https://git-scm.com/download/win

B3. Abra o Docker Desktop e espere o ícone da baleia ficar parado, sem a mensagem "starting".

B4. Abra o **Git Bash** e confira se o Docker responde:

```
docker --version
```

```
docker compose version
```

B5. Baixe o projeto:

```
cd ~
```

```
git clone https://github.com/Conecta-Vida/Docker-Teste.git
```

```
cd Docker-Teste
```

> Todos os comandos deste manual são executados **no Git Bash**, dentro da pasta `~/Docker-Teste`. No PowerShell os scripts `.sh` não funcionam.

Parte C: subir o projeto

C1. Atualize o projeto. Faça isso sempre antes de apresentar:

```
cd ~/Docker-Teste && git pull
```

C2. Se já rodou uma versão antiga antes, limpe tudo:

```
./infra/swarm/seminario.sh derrubar --apagar-dados
```

C3. Suba o cluster:

```
./infra/swarm/seminario.sh subir
```

O script faz tudo sozinho:

1. Ativa o Docker Swarm.
2. Constrói as 4 imagens. **Na primeira vez leva de 10 a 15 minutos**, porque baixa o Java, o Node e o Flutter. Depois fica rápido.
3. Cria o registry local e envia as imagens para ele.
4. Cria os workers `worker-1` e `worker-2` (Docker-in-Docker).
5. Conecta os workers ao manager e etiqueta os nós.
6. Faz o deploy: `docker stack deploy -c docker-compose.yml conectavida`.

> **O que é o registry?** Um "depósito de imagens" dentro do cluster (`127.0.0.1:5000`). As imagens são construídas no manager e enviadas para lá; os workers, que têm um Docker próprio e vazio, baixam as imagens dele. Para ver o conteúdo: `curl http://localhost:5000/v2/_catalog`.

Git Bash

Registry local: as imagens que os workers baixam

```
gusta@Gustavo UCRT64 ~/Docker-Teste
$ curl http://localhost:5000/v2/_catalog
{"repositories":["conectavida/admin-api","conectavida/admin-web","conectavida/mobile-api","conectavida/mobile-web"]}
```

C4. Espere cerca de **1 minuto** e confira:

```
./infra/swarm/seminario.sh status
```

Está pronto quando:

- os 3 nós aparecem como `Ready`;
- todos os serviços mostram réplicas completas (`1/1`, `2/2`, `3/3`).

Git Bash

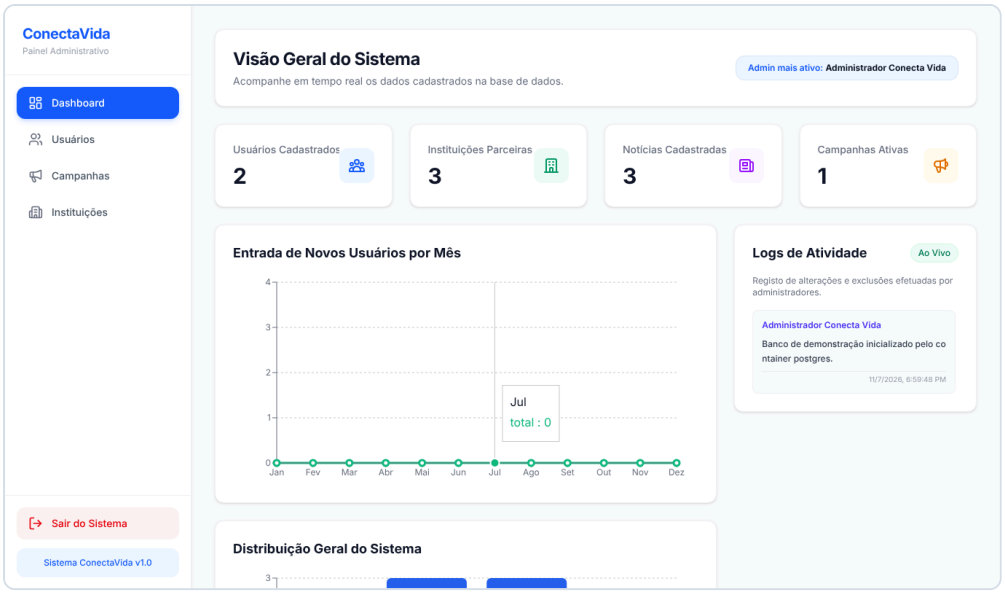
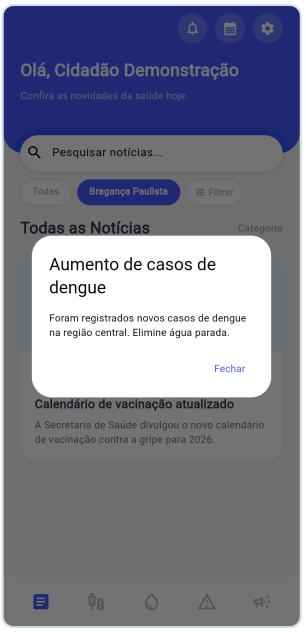
Requisito 2 · Cluster Swarm com 3 nós: 1 manager + 2 workers Docker-in-Docker

```
gusta@Gustavo UCRT64 ~/Docker-Teste
$ docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
nzj51s2vykomiqh6udde6zsm9 *   vm         Ready     Active         Leader           29.9.2
1e9vyv4txumy6cbihv80fijaw     worker-1   Ready     Active                          29.9.3
7v42ju3l2xjjl1pmm4yu4tkdr     worker-2   Ready     Active                          29.9.3
```

C5. Abra no navegador:

O quê	Endereço	Login
Visualizer	http://localhost:8085	(sem login)
Painel do Cluster (diferencial)	http://localhost:8090	(sem login)
Painel admin	http://localhost:8088	`admin@conectavida.com` / `admin123`
App do cidadão	http://localhost:8089	`cidadao@conectavida.com` / `cidadao123`
Documentação das APIs	http://localhost:8080/swagger-ui.html	(sem login)

> Para o app parecer de celular: aperte **F12** no Chrome e clique no ícone de celular (Ctrl+Shift+M).

Painel admin	App do cidadão
	

Parte D (opcional): usar os dados reais do Supabase

O banco começa com dados de demonstração. Para trazer os dados que estavam no Supabase:

D1. Pegue a senha atual do banco em **Supabase → Project Settings → Database**.

D2. Com a stack rodando, no Git Bash:

```
export SUPABASE_DB_URL='postgresql://postgres.pborxjuuexiqollesfxw@aws-1-sa-east-1.pooler.supabase.com:5432/postgres'
```

```
export SUPABASE_DB_PASSWORD='cole-a-senha-aqui'
```

```
./infra/postgres/importar-supabase.sh
```

D3. Para os **logins antigos** funcionarem, o salt precisa ser o mesmo de antes. Ele está no `app.security.salt` dos `application.properties` antigos. Antes do `subir`, rode:

```
export CONECTA_API_SALT='valor-do-salt-antigo'
```

> O script só **lê** do Supabase e não altera nada lá. Se apagar os dados com `derrubar --apagar-dados`, rode a importação de novo.

Parte E: preparar a apresentação (5 minutos antes)

E1. Confirme que tudo está no ar (Parte C4).

E2. Organize a tela:

- **Navegador** com 3 abas: Visualizer (8085), painel (8088) e app (8089).
- **VS Code** com estes arquivos abertos: `admin-api/Dockerfile`, `admin-web/Dockerfile` e `docker-compose.yml`.
- **Git Bash** com a fonte grande (Ctrl + roda do mouse).

E3. Faça login no painel e no app antes, para não perder tempo.

Parte F: roteiro da apresentação (cerca de 14 minutos)

F1. Abertura (1 min)

> "O Conecta Vida centraliza informações de saúde pública: o gestor publica campanhas e informativos pelo painel web e o cidadão recebe pelo aplicativo. Hoje vamos mostrar como colocamos o sistema inteiro em containers e orquestramos com o Docker Swarm."

F2. Requisito 1: Dockerfiles (3 min, no VS Code)

Abra o `admin-api/Dockerfile` e explique:

> "Este Dockerfile é **multi-stage**. O primeiro estágio, `FROM eclipse-temurin:21-jdk-alpine AS build`, tem o JDK completo e compila o projeto com o Gradle. O segundo, `FROM eclipse-temurin:21-jre-alpine`, tem só o Java de execução e copia **apenas o `app.jar`**. A imagem final fica sem compilador nem código-fonte, roda em **Alpine Linux**, com um **usuário sem root** e um **healthcheck** que o Swarm usa para saber se a réplica está saudável."

Abra o `admin-web/Dockerfile`:

> "O front segue a mesma ideia: o Node compila o React e o Nginx entrega o site. A imagem final tem cerca de 74 MB. O Nginx também repassa as chamadas `/api` para a API dentro do cluster."
>
> "No total são 4 imagens nossas: as duas APIs, o painel e o app em versão web."

admin-api/Dockerfile

Requisito 1 · Dockerfile multi-stage: estágio 1 compila (JDK), estágio 2 só executa (JRE + app.jar)

```
# ---------- Estágio 1: build (JDK 21 em Alpine Linux + Gradle Wrapper) ----------
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /workspace

# 1) Arquivos de build primeiro: a camada de dependências fica em cache
COPY gradlew settings.gradle build.gradle ./
COPY gradle ./gradle
COPY core/build.gradle core/
COPY admin-api/build.gradle admin-api/
COPY mobile-api/build.gradle mobile-api/
RUN chmod +x gradlew \
    && ./gradlew --no-daemon -q :admin-api:dependencies --configuration runtimeClasspath > /dev/null

# 2) Código-fonte: o módulo compartilhado :core e o módulo da API
COPY core/src core/src
COPY admin-api/src admin-api/src
RUN ./gradlew --no-daemon :admin-api:bootJar -x test

# ---------- Estágio 2: runtime (somente JRE 21 em Alpine Linux + app.jar) ----------
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Usuário sem privilégios de root
RUN addgroup -S conectavida && adduser -S conectavida -G conectavida
COPY --from=build --chown=conectavida:conectavida /workspace/admin-api/build/libs/app.jar app.jar
USER conectavida

ENV SERVER_PORT=8080 \
    JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
EXPOSE 8080

HEALTHCHECK --interval=15s --timeout=5s --start-period=60s --retries=5 \
  CMD wget -qO- http://127.0.0.1:8080/actuator/health/liveness || exit 1

ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

admin-web/Dockerfile

Requisito 1 · Front-end: Node compila o React, Nginx (Alpine) entrega o site

```
# ---------- Estágio 1: build (Node.js 22 em Alpine Linux) ----------
FROM node:22-alpine AS build
WORKDIR /app

# Dependências primeiro (camada em cache enquanto package*.json não mudar)
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund

# Código-fonte e build de produção (TypeScript + Vite -> /app/dist)
COPY . .
# Vazio = mesma origem: as rotas do admin V2 já começam com /api e o Nginx as encaminha
ARG VITE_API_URL=
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

# ---------- Estágio 2: runtime (Nginx em Alpine Linux) ----------
FROM nginx:1.27-alpine
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
HEALTHCHECK --interval=15s --timeout=3s --retries=3 \
  CMD wget -qO- http://127.0.0.1/healthz || exit 1
```

Git Bash

Requisito 1 · As 4 imagens do projeto, em Alpine Linux e sem root

```
gusta@Gustavo UCRT64 ~/Docker-Teste
$ docker images | grep conectavida
REPOSITORY                          TAG                 SIZE
conectavida/mobile-web              1.0                 132MB
conectavida/admin-web               1.0                 74.7MB
conectavida/mobile-api              1.0                 397MB
conectavida/admin-api               1.0                 425MB

gusta@Gustavo UCRT64 ~/Docker-Teste
$ docker run --rm --entrypoint sh conectavida/admin-api:1.0 -c 'grep PRETTY /etc/os-release; java -version; whoami'
PRETTY_NAME="Alpine Linux v3.24"
openjdk version "21.0.13" 2026-10-20 LTS
conectavida
```

F3. Iniciar o roteiro guiado

No Git Bash:

```
./infra/swarm/seminario.sh apresentar
```

O script mostra **em amarelo o que falar**, executa cada demonstração e espera você apertar **Enter**.

F4. Requisito 2: Swarm (passos 1 e 2 do script, 2 min)

- **Passo 1, os nós:**

  > "Temos 3 nós. O **manager** coordena o cluster: decide onde cada container roda e recria o que falhar. Os **workers** executam os containers. Usamos **Docker-in-Docker**: cada worker é um container que roda um Docker dentro dele, simulando outra máquina."

- **Passo 2, o Visualizer:**

  > "O banco fica **fixo no manager** (`constraint: node.role == manager`), porque guarda dados. As APIs e os sites não guardam estado, então o Swarm **espalha as cópias** entre os nós. Caixas da mesma cor são réplicas do mesmo serviço."

🎬 **Vídeo:** os 3 nós e o Visualizer (21 s)

docker-compose.yml

O banco fica fixo no manager (constraint) e guarda os dados num volume

```yaml
  # 1. Banco de dados relacional ------------------------------------
  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-conectavida}
      POSTGRES_USER: ${POSTGRES_USER:-conectavida}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-conectavida}
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    configs:
      - source: db-schema
        target: /docker-entrypoint-initdb.d/01-schema.sql
      - source: db-seed
        target: /docker-entrypoint-initdb.d/02-seed.sql
    healthcheck:
      # $$ = variável lida dentro do container (não pelo compose)
      test: ["CMD-SHELL", "pg_isready -U \"$${POSTGRES_USER}\" -d \"$${POSTGRES_DB}\""]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - conectavida-net
    deploy:
      labels:
        conectavida.camada: "dados"
      replicas: 1
      placement:
        constraints: [node.role == manager]
      restart_policy:
        condition: on-failure
```

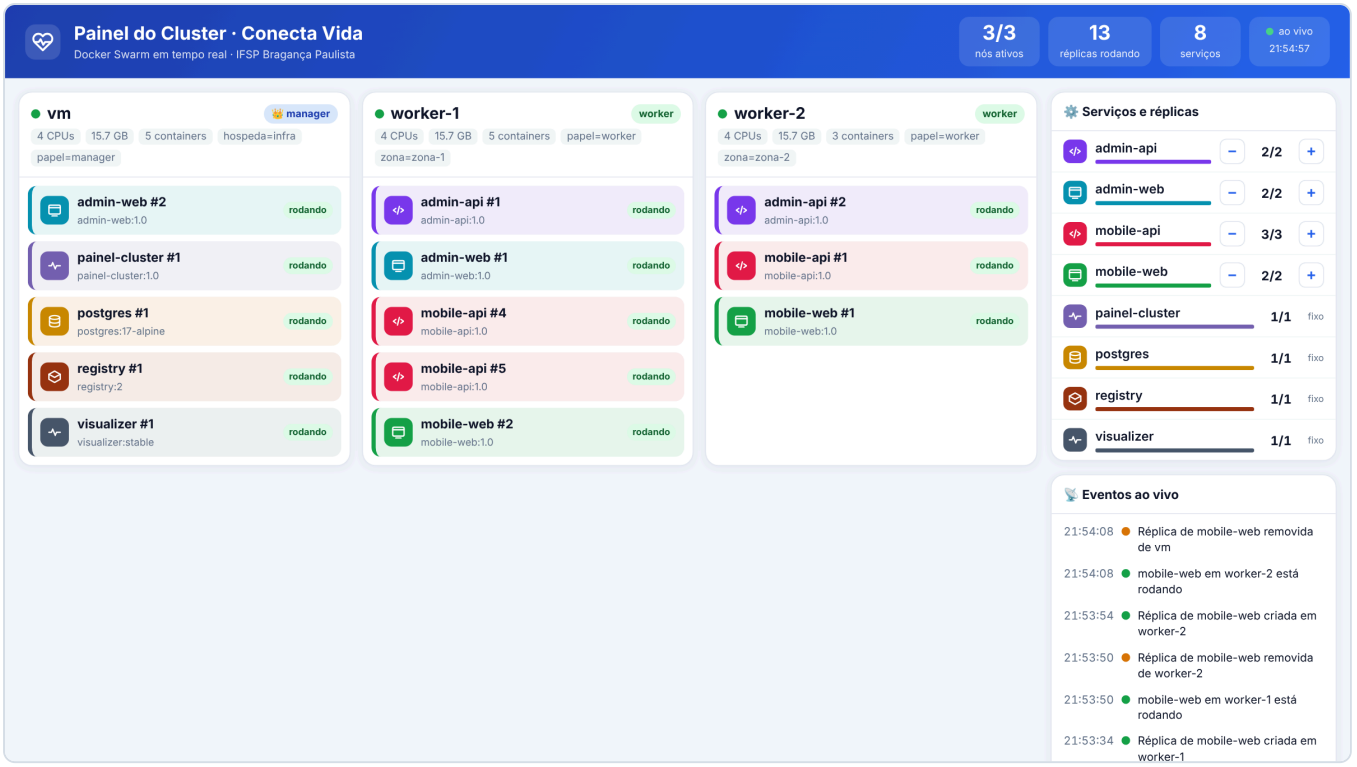
F4b. Nosso diferencial: Painel do Cluster Conecta Vida (2 min)

O script abre http://localhost:8090 no passo 2. Mostre lado a lado com o Visualizer oficial.

> "Além do Docker Visualizer, criamos o **nosso próprio visualizador**, o Painel do Cluster Conecta Vida. Ele roda como mais um serviço da stack, com Dockerfile próprio, e conversa com a **API do Docker** pelo socket do manager. Mostra os 3 nós, as réplicas de cada serviço com ícones e cores, um **feed de eventos ao vivo** e tem **botões + e −** que escalam o serviço direto pela tela."

Demonstração:

1. Clique em **+** na `mobile-api` duas ou três vezes. As réplicas novas aparecem como `starting` e depois `rodando`, e o feed registra *"Réplica de mobile-api criada em worker-1"*.
2. Mostre que o **Visualizer oficial** (8085) também mudou: o painel altera o Swarm de verdade.
3. Tente o **+** no `postgres`: não há botão, ele é **fixo** (banco não é escalado).
4. Clique em **−** para voltar.



🎬 **Vídeo:** Painel do Cluster: escala pelos botões e queda do worker-2 registrada no feed (115 s)

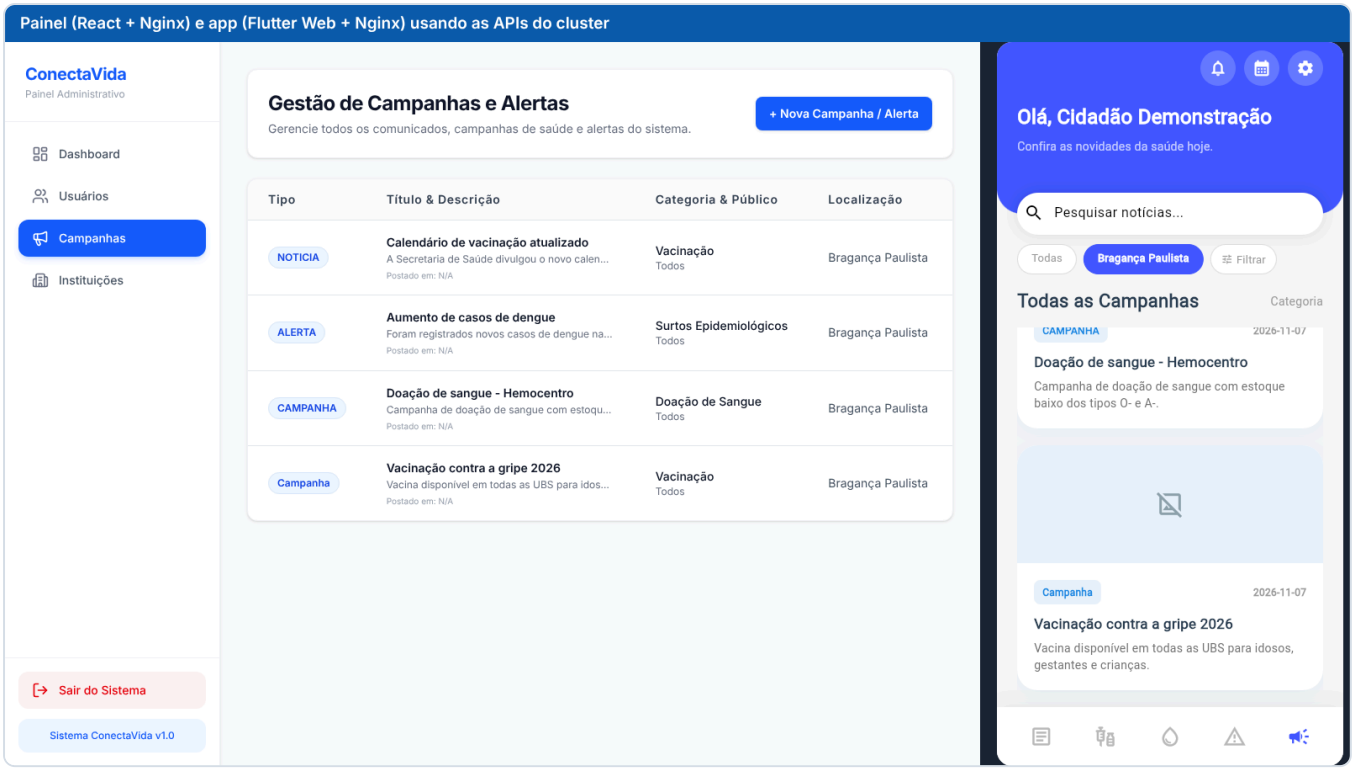
> Os botões fazem o mesmo que `docker service scale`. No próximo `docker stack deploy` vale de novo o número do `docker-compose.yml`.

F5. O sistema funcionando (passo 3, 2 min)

- No **painel**, mostre o dashboard e, em **Campanhas**, crie uma campanha nova (ex.: "Vacinação contra a gripe").
- No **app**, atualize a página e mostre a campanha aparecendo para o cidadão, que pode se inscrever.

> "As duas telas conversam com APIs que estão rodando em vários nós do cluster, ao mesmo tempo."

🎬 **Vídeo:** login no painel e no app, campanha criada no painel aparecendo no app (65 s)



F6. Requisito 3: Compose e variação de réplicas (passos 4 e 5, 3 min)

Antes do passo 4, mostre no VS Code o `docker-compose.yml`:

> "Isto é **infraestrutura como código**: os 7 serviços, a rede overlay, o volume do banco, os limites de memória e o número de réplicas estão descritos neste arquivo. Vejam a linha `replicas: ${MOBILE_API_REPLICAS:-3}` : o padrão são 3 cópias."

- **Passo 4:** o script reaplica o arquivo com 6 réplicas.

  > "Olhem o Visualizer: as caixas da mobile-api passaram de 3 para 6, distribuídas entre os nós."

- **Ainda no passo 4:** no próximo Enter, o script volta para 2 réplicas e as caixas somem.
- **Passo 5:** o script usa `docker service scale`.

  > "Também dá para escalar direto pelo terminal. O arquivo continua sendo a referência: no próximo deploy, vale o que está nele."

docker-compose.yml

Requisito 3 · Infraestrutura como código: o número de réplicas está declarado no arquivo

```
92   # 3. API do aplicativo do cidadão --------------------------------
93   mobile-api:
94     image: ${REGISTRY:-127.0.0.1:5000}/conectavida/mobile-api:${TAG:-1.0}
95     environment:
96       <<: *api-env
97       SERVER_PORT: "8081"
98     ports:
99       - "8081:8081"
100    depends_on:
101      - postgres
102    networks:
103      - conectavida-net
104    deploy:
105      labels:
106        conectavida.camada: "backend-mobile"
107      replicas: ${MOBILE_API_REPLICAS:-3}   # número de cópias (variável MOBILE_API_REPLICAS)
108      update_config: *rolling-update
109      restart_policy:
110        condition: on-failure
111        delay: 10s
112      resources:
113        limits:
114          memory: 512M
115        reservations:
116          memory: 128M
```

🎬 **Vídeo:** mobile-api de 3 para 6 e de volta para 2 pelo docker-compose.yml (66 s)

🎬 **Vídeo:** admin-api de 2 para 4 com docker service scale (46 s)

F7. Extra: alta disponibilidade (passo 6, 1 min)

> "E se uma máquina cair?"

O script desliga o `worker-2`. Em cerca de 30 segundos ele fica vermelho no Visualizer e as réplicas dele reaparecem nos outros nós. Atualize o painel para mostrar que o sistema continua funcionando. No próximo Enter o worker é religado.

🎬 **Vídeo:** worker-2 cai, as réplicas migram, e depois ele volta (75 s)

F8. Fechamento (passo 7, 30 s)

> "Atendemos os três requisitos: **Dockerfiles** para 5 serviços, **cluster Swarm com 3 nós em Docker-in-Docker**, e **Docker Compose como infraestrutura como código**, variando réplicas e acompanhando tudo pelo **Docker Visualizer** e pelo nosso **Painel do Cluster**, o diferencial do grupo. Obrigado!"

Parte G: divisão para 3 apresentadores

Quem	Partes	Tempo	Fica no controle de
Pessoa 1	Abertura e **Requisito 1: Dockerfiles** (F1 e F2)	~4 min	VS Code (mostra `admin-api/Dockerfile` e `admin-web/Dockerfile`)
Pessoa 2	**Requisito 2: Swarm**, **diferencial (Painel do Cluster)** e sistema funcionando (F3, F4, F4b e F5)	~5 min	Git Bash (`apresentar`, passos 1 a 3) e navegador (Painel do Cluster, painel admin e app)
Pessoa 3	**Requisito 3: Compose e réplicas**, queda de máquina e fechamento (F6, F7 e F8)	~4 min	VS Code (`docker-compose.yml`) e Git Bash (passos 4 a 7)

Dicas:

- Um só computador e um só terminal: quem está falando pede para o colega apertar Enter, ou assume o teclado na sua vez.
- A Pessoa 2 já deixa o painel e o app logados antes de começar.
- Se perguntarem sobre o registry: "É um depósito de imagens dentro do cluster. Construímos as imagens no manager e os workers baixam de lá." Para mostrar: `curl http://localhost:5000/v2/_catalog`.

Parte H: comandos avulsos (se quiser fazer na mão)

Para	Comando
Ver nós, serviços e réplicas	`./infra/swarm/seminario.sh status`
Mudar réplicas pelo compose	`./infra/swarm/seminario.sh replicas mobile-api 6`
Escalar pelo terminal (6/4/2)	`./infra/swarm/seminario.sh escalar`
Escalar um serviço específico	`docker service scale conectavida_mobile-api=6`
Desligar o worker-2	`./infra/swarm/seminario.sh falha`
Religar o worker-2	`./infra/swarm/seminario.sh recuperar`
Ver os nós	`docker node ls`
Ver onde cada réplica está	`docker stack ps conectavida`
Ver o log de uma API	`docker service logs conectavida_admin-api`

Parte I: se algo der errado

Problema	Solução
O Windows pergunta "com qual aplicativo abrir o .sh"	Você está no PowerShell. Use o **Git Bash**.
`docker: command not found` ou erro de conexão	Abra o Docker Desktop e espere ficar pronto.
Alguma caixa vermelha logo depois do `subir`	Normal: as APIs reiniciam até o banco ficar pronto. Espere 1 minuto.
Painel diz "Não foi possível conectar à API"	Espere 1 minuto e confira com `./infra/swarm/seminario.sh status`.
Os workers não entram no cluster	Rode `docker swarm leave --force` e depois `./infra/swarm/seminario.sh subir`.
Depois do `falha`, o worker não volta	`./infra/swarm/seminario.sh recuperar`
Quer recomeçar do zero	`./infra/swarm/seminario.sh derrubar --apagar-dados` e depois `./infra/swarm/seminario.sh subir`
Importação do Supabase: "password authentication failed"	Usuário `postgres.pborxjuuexiqollesfxw`, porta 5432 e senha atual em `SUPABASE_DB_PASSWORD`.
Nada funciona na hora da apresentação	Use os vídeos e fotos da **Parte K** (plano B).

Parte J: depois da apresentação

Para desligar tudo e liberar memória:

```
./infra/swarm/seminario.sh derrubar
```

Para sair do modo Swarm também:

```
docker swarm leave --force
```

Mais detalhes técnicos (cada Dockerfile e o compose explicados bloco a bloco): `documentacao/DOCKER_SWARM.md`.

Parte K: fotos e vídeos (plano B e slides)

Gravados com o projeto rodando num cluster de 3 nós (manager + 2 workers Docker-in-Docker). Arquivos em `documentacao/midia/`.

Vídeos

Vídeo	Mostra	Usar em
video-1-cluster-3-nos.mp4	`docker node ls` e o Visualizer com os 3 nós	Requisito 2
video-2-replicas-pelo-compose.mp4	mobile-api 3 → 6 → 2 réplicas pelo `docker-compose.yml`	Requisito 3
video-3-escala-pelo-terminal.mp4	admin-api 2 → 4 → 2 com `docker service scale`	Requisito 3
video-4-queda-de-maquina.mp4	worker-2 cai e as réplicas migram para os outros nós	Extra
video-5-sistema-funcionando.mp4	painel cria uma campanha e ela aparece no app	Sistema
video-6-painel-do-cluster.mp4	**Diferencial:** Painel do Cluster escalando pelos botões e registrando a queda do worker-2	Diferencial

> No GitHub, clique no vídeo e depois em **View raw** (ou no botão de download) para baixar o `.mp4` e colocar nos slides.

Fotos

Foto	Mostra
01-dockerfile-admin-api.png	Dockerfile multi-stage da admin-api
02-dockerfile-admin-web.png	Dockerfile do painel (Node + Nginx)
03-imagens-alpine.png	As 4 imagens, Alpine Linux e usuário sem root
04-docker-node-ls.png	Os 3 nós do Swarm
05-visualizer-3-nos.png	Visualizer com os serviços distribuídos
06-compose-replicas.png	Réplicas no docker-compose.yml
07-compose-postgres.png	Banco fixo no manager (constraint)
08-registry.png	Registry local com as imagens
09-painel-login.png	Login do painel admin
10-painel-dashboard.png	Dashboard
11-painel-campanhas.png	Campanhas e alertas
12-painel-usuarios.png	Usuários
13-painel-instituicoes.png	Instituições de saúde
14-app-login.png	Login do app
15-app-inicio.png	App mostrando o alerta ao cidadão
16-sistema-funcionando.png	Campanha criada no painel aparecendo no app
17-painel-do-cluster.png	**Diferencial:** Painel do Cluster Conecta Vida